Graph cross-platform measurement

Created-at: 2026-09-06 12:50 +02:00.

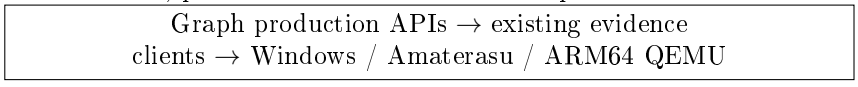
Scope and acceptance Execute the existing Graph search and algorithm clients on Windows, native Amaterasu and ARM64 Docker/QEMU on Amaterasu. Reuse canonical checkouts and target caches. Production and Cargo dependencies remain unchanged. Preserve checksums, correctness results, raw samples, median/IQR/worst, binary/source hashes and explicit unsupported metrics.

Workloads Existing complete 48-cell informed Berge matrix: grids, weighted maps, corridor and transport; BFS/Dijkstra on path65k, grid256, STS1023, random1e6 and map512; bounded paths/cycles. Add existing reachability, components, rank-2 and Levi clients on path65k and STS511, plus independent 64-query batches at 1/2/4 threads on STS511. Rank-2 on non-rank-2 input remains explicitly not applicable. Three independent process runs per platform, two warmups and nine retained samples per cell. No representation expansion experiment is added.

Affected paths scripts/performance_evidence/campaign.py and test_campaign.py; this plan bundle; reports/cross-platform-benchmarks/2026-09-06/* including raw logs, JSON/CSV and SVG; reports/2026-09-06-cross-platform-benchmarks.md; generated SYMBOLS.md only if required by the canonical index. Protected items touched: empty. No interface changes.

Tests First validate parser against malformed rows, missing samples, drift and failures. Build pinned release clients and run their non-ignored correctness tests on all three platforms before timing. Compare invariant results and search work across runs/platforms; compare BFS and Dijkstra costs/hops, A* optimal costs. Unsupported CPU values remain missing, never zero. Source tree clean or explicitly recorded measurement-driver-only changes.

Budget and ETA About 45–75 minutes: existing 48-cell baseline has 2 warmups plus 9 batches targeting 250ms each; three runs of 59 cells on three hosts, with local and remote campaigns on separate machines. QEMU does not run concurrently with native measurements or website builds. Compile four jobs, 16GiB memory cap; each process 180s timeout, report any overrun before changing scope. Stop on checksum drift or failure. Observe progress for every process and every ten seconds. No unbounded enumerations; existing max-depth/result/expansion limits stay authoritative.

Risks Host noise, QEMU overhead, timer quantization and build interference. Record ambient CPU, exact compiler/lock/source/runtime image and excluded runs. Memory is whole-process peak, not per operation. Samples are batch means, not request tails. ARM results are emulated, not hardware claims.

Rollback Revert campaign scripts/evidence commit, remove only named transient build staging after successful extraction, preserve reusable caches and all production sources.